
Assignment 2: Reinforce, AC and A2C

Rejdi Danaj s3988651¹ Manuel Pérez Belizón s4416880² Harshad Sansanwal s3966003³

Abstract

We compare three policy gradient methods—REINFORCE, Actor-Critic (AC), and Advantage Actor-Critic (A2C)—on the CartPole environment, focusing on stability, variance, and architectural sensitivity. The implementation of each algorithm is complemented with theoretical bases and the respective algorithmic pseudocode. Empirically, REINFORCE outperforms both AC and A2C in terms of consistency and convergence time, primarily because Monte Carlo sampling is computationally inexpensive for this particular environment. AC experiences high bias and convergence issues due to early variance introduced by the critic network, while A2C with normalization significantly improves performance but remains sensitive to hyperparameter tuning. These results highlight the robustness of Monte Carlo approaches in low-cost settings and the importance of tuning in bootstrapped methods.

1. Introduction

Reinforcement Learning (RL) provides a large array of tools for tackling different problems. Agents are trained to perform sequential decisions by directly interacting with the environment. This paradigm allows for large flexibility in training approaches. Among this diverse group of RL methods, policy gradient algorithms play a central role in enabling agents to draw on prior experience and learn from their previous successes. This assignment focuses on the theoretical basis, implementation, and experimentation of three fundamental policy gradient algorithms: REINFORCE, ACTOR-CRITIC (AC), and ADVANTAGE ACTOR-CRITIC (A2C).

We begin with the theoretical foundation of these three algorithms. These algorithms are then implemented and evaluated through a series of experiments conducted in the CartPole-v1 environment, with a focus on analyzing their performance in terms of stability, convergence, and variance.

2. Theory

When working with environments with a continuous state space, Q-learning and DQN fall short since we cannot use the *argmax* approach for selecting actions. Instead, we use policy-based methods that directly learn a parameterized policy. These model-free approaches select actions without relying on a value function, enabling support for continuous or stochastic action spaces (*basis the axiom of choice*).

2.1. Vanilla Policy Loss

To define policy selection, we start with a parameterized policy π_θ , where θ are learnable parameters. From this policy, we sample a trajectory (τ) from the environment. The goal is to optimize the policy to favor high-reward trajectories. To do this, we define a quality function:

$$J(\theta) = V^\pi(s_0) \quad [\text{with } s \sim \mathcal{D}]$$

We aim to maximize $J(\theta)$ via gradient ascent:

$$\nabla_\theta J(\theta) = \nabla_\theta V^\pi(s_0)$$

$$\theta_{t+1} = \theta_t + \alpha \cdot \nabla_\theta J(\theta) \quad [\alpha \leftarrow \text{learning rate}]$$

Since we don't know the best actions directly, we use an estimated Q-function $\hat{Q}$ to update the policy. To stabilize learning and avoid over-committing to specific actions, we adjust the gradient using the log-likelihood trick:

$$\theta_{t+1} = \theta_t + \alpha \hat{Q}(s, a) \cdot \frac{\nabla \pi_{\theta_t}(a|s)}{\pi_{\theta_t}(a|s)} \quad [\text{Since } \nabla \log f(x) = \frac{\nabla f(x)}{f(x)}]$$

Which gives the standard REINFORCE update:

$$\theta_{t+1} = \theta_t + \alpha \hat{Q}(s, a) \cdot \nabla_\theta \log \pi_\theta(a|s)$$

From this, we define the vanilla policy loss function over the policy distribution as:

$$\mathcal{L}(\theta) = \mathbb{E}_{s \sim \mathcal{D}} [\mathbb{E}_{a \sim \pi_\theta(a|s)} [-Q(s, a) \cdot \nabla_\theta \log \pi_\theta(a|s)]] \quad (1)$$

[The negative is added to the loss function to denote maximization]

2.2. REINFORCE

Motivated by our derivation of the policy loss function, we can now uncover the REINFORCE algorithm. Given a differentiable policy $\pi_\theta(a|s)$, we initialize random parameters

θ as function for the policy. Now we deploy Monte-Carlo sampling, by using initial policy we draw random trajectories τ from the environment. For each trajectory we calculate the discounted reward across steps and update the initial θ using gradient ascent. We chose a small ϵ as the threshold, below which we consider the policy function to be converging. This approach because of episodic randomness has low bias in results, but suffers high variance.

2.3. Actor Critic

To address the high variance of Reinforce algorithm, we now introduce Actor-Critic (AC) method. This approach involves policy-based (actor) and value-based (critic) techniques. The actor, parametrized by θ , learns the policy $\pi_\theta(a|s)$, while the critic, parametrized by ϕ , estimates the value function $V_\phi(s)$ helping guide the policy. We start by randomly initializing parameters for θ & ϕ . At each iteration, we sample a trajectory from the environment according to the current policy. To estimate the return from a given state-action pair, we compute the n-step return $\hat{Q}_n(s_t, a_t)$, which consists of two parts: the cumulative discounted rewards over $t + k$ steps, and an estimate of the value at the steps $t + n$ using the critic function $V_\phi(s_{t+n})$. Finally we first update the critic by using the least mean squared error loss function with the estimated return $\hat{Q}_n$ and the value function $V_\phi(s_t)$ across the trajectory. The actor is updated using the policy loss function *equation (1)*, with replacing the Q-value with our estimated return $\hat{Q}_n$.

2.4. Q-Network over Q-Learning loss

One would think that in our actor critic method why can we not just imply our Q-Learning loss derived using the *Bellman Equation* as a critic? If we do use the Q-Learning loss, we are in fact selecting the best possible action say a_t^* at any particular state say s_t . This a_t^* is being selected from the complete action space possible at s_t , let this space be denoted by a set say $A = \{a_t^1, a_t^2, \dots, a_t^n\}$. While in a policy based method like actor critic, we are interested in selecting a_t^* at s_t from the action space that falls under a selected policy say π_θ , where this set would be a subset of A.

2.5. Advantage Actor Critic

Another algorithm to reduce the high variance is the advantage actor critic method or A2C. This method utilizes an advantage function $A(s_t, a_t)$, calculated by subtracting the Value $V(s_t)$ from the state-action estimate value $Q(s_t, a_t)$. This allows us to estimate how better a particular action is comparative to the probability of a particular state. The Algorithm for A2C stays the same as for the actor critic method. We initialize the same parameters and follow the same steps until the estimation of the Q-value using the $n - step$ V-value. Now we calculate the Advantage for the

Algorithm 1 Actor-Critic with TD bootstrapping

Input : Discount factor γ , learning rates α , estimation steps n , max steps $maxSteps$

Initialize : Policy parameters θ , value parameters ϕ

```

1 for each run do
2   Initialize environment;  $steps \leftarrow 0$ 
3   while  $steps < maxSteps$  do
4     Reset environment to state  $s$ ; episode trace  $\tau \leftarrow \emptyset$ ;
        $done \leftarrow False$ 
5     while  $not\ done\ and\ steps < maxSteps$  do
6       Sample action  $a \sim \pi_\theta(a|s)$ 
7       Take action  $a$ , observe reward  $r$ , next state  $s'$ ,
         and termination signal  $done$ 
8       Store  $(s, a, r)$  in episode trace  $\tau$ 
9        $s \leftarrow s'$ ;  $steps \leftarrow steps + 1$ 
10    for each step  $t$  in trace  $\tau$  do
11      Compute  $n$ -step return:
          
$$\hat{Q}_n(s_t, a_t) = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V_\phi(s_{t+n})$$

12      Normalize returns:
          
$$\hat{Q}_n \leftarrow \frac{\hat{Q}_n - \text{mean}(\hat{Q}_n)}{\text{std}(\hat{Q}_n) + \epsilon}$$

13      Compute critic loss and update:
          
$$\phi \leftarrow \phi - \nabla_\phi \sum_t (\hat{Q}_n(s_t, a_t) - V_\phi(s_t))^2$$

14      Compute actor gradient and update:
          
$$\theta \leftarrow \theta + \alpha \cdot \sum_t [\hat{Q}_n(s_t, a_t) \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)]$$


```

complete trajectory by subtracting the estimated V_ϕ from the estimated $\hat{Q}_n$. Then we finally update our ϕ parameters using $\hat{A}_n$ which is similar to actor critic. What changes is the updating of θ parameters, as now we use the below gradient descent step:

$$\theta \leftarrow \theta + \alpha \cdot \sum_t [\hat{A}_n(s_t, a_t) \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)]$$

The reason we have updated our loss policy to this is fairly simple. Consider the vanilla policy loss *equation (1)*, here we used $Q(s, a)$ to help optimize towards best actions. But by using advantage we are able to reduce variance for over estimation and hence we use $A(s, a)$ to help estimate the best action policy.

$$\mathcal{L}(\theta) = \mathbb{E}_{s \sim \mathcal{D}} \left[\mathbb{E}_{a \sim \pi_\theta(a|s)} \left[-\hat{A}_n(s, a) \cdot \nabla_\theta \log \pi_\theta(a|s) \right] \right]$$

Algorithm 2 Advantage Actor-Critic with TD bootstrapping

Input : Discount factor γ , learning rates α , estimation steps n , max steps $maxSteps$

Initialize : Policy parameters θ , value parameters ϕ

```

15 for each run do
16   Initialize environment;  $steps \leftarrow 0$ 
17   while  $steps < maxSteps$  do
18     Reset environment to state  $s$ ; episode trace  $\tau \leftarrow \emptyset$ ;
        $done \leftarrow False$ 
19     while not  $done$  and  $steps < maxSteps$  do
20       Sample action  $a \sim \pi_\theta(a|s)$ 
21       Take action  $a$ , observe reward  $r$ , next state  $s'$ ,
       and termination signal  $done$ 
22       Store  $(s, a, r)$  in episode trace  $\tau$ 
23        $s \leftarrow s'$ ;  $steps \leftarrow steps + 1$ 
24     for each step  $t$  in trace  $\tau$  do
25       Compute  $n$ -step return:

           
$$\hat{Q}_n(s_t, a_t) = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V_\phi(s_{t+n})$$


26       Normalize returns:

           
$$\hat{Q}_n \leftarrow \frac{\hat{Q}_n - \text{mean}(\hat{Q}_n)}{\text{std}(\hat{Q}_n) + \epsilon}$$


27       Compute critic loss and update:

           
$$\phi \leftarrow \phi - \nabla_\phi \sum_t (\hat{Q}_n(s_t, a_t) - V_\phi(s_t))^2$$


28       Compute normalized advantages:

           
$$A(s_t, a_t) = \frac{\hat{Q}_n(s_t, a_t) - V_\phi(s_t)}{\text{std}(\hat{Q}_n(s_t, a_t) - V_\phi(s_t)) + \epsilon}$$


29       Compute actor gradient and update using advantages:

           
$$\theta \leftarrow \theta + \alpha \cdot \sum_t [A(s_t, a_t) \cdot \nabla_\theta \log \pi_\theta(a_t|s_t)]$$

    
```

3. Experiment

This section provides a detailed description of the implementation and subsequent experimentation of the three main algorithms introduced in the theory section. A detailed breakdown of both the implementation and the experimental design is provided below.

3.1. implementation

REINFORCE To implement the Reinforce algorithm, we used a fully-connected neural network as the policy network. Action selection is performed by passing the state through the network to obtain action probabilities. These probabilities are used to create a categorical distribution from which an action is sampled, introducing exploration. During each episode, we collect the log-probability of the selected action and the rewards. Before the end of the episode, we compute the returns using the rewards, we normalize those returns, and calculate the loss with the returns and the log-probability of the selection.

We run the algorithm $num_runs = 5$ times for 10^6 steps and take the average of all the runs to minimize randomness.

AC Following the steps for Reinforce algorithm, for AC algorithm we introduce a critic network. This is another additional fully-connected neural network that will estimate the value function for n -steps and the full trajectories. As mentioned previously, the primary reason for adding the critic is to reduce the variance in the results. As mentioned previously AC follows similar steps as reinforce algorithm. But enhance the Q-value calculations by introducing the critic function in the mix which estimates V-values for n -steps here. Additionally, we now also update the parameters for our critic function using the Q-value and estimated full trace critic function. In addition to the hyperparameters tested in the Reinforce algorithm, we have 1 additional hyperparameter here $depth = n$. We chose three different values for $n \in [5, 15, 20]$. These values were arbitrarily chosen, though loosely based on suggestion presented by https://dbtml.github.io/rl_section.html. We considered both small and medium n values instead of high values, as we understand that the CartPole environment does still have a finite action state. Both repetition and max number of steps were kept similar as the Reinforce algorithm.

A2C The A2C implementation follows directly from AC. The actor and critic are handled as different networks and not as a multihead one. This was done primarily to avoid inefficient loss value distribution through the network. This implementation shares the same hyperparameters as the AC implementation, namely, α , γ , $depth$, and network architecture. Advantage was estimated after a full sample, where the output was then normalized or used directly, depending on a boolean parameter set as a hyperparameter. Again, as with AC, both the critic and actor policy networks are updated accordingly after each sampling episode. The rewards are then stored and paired with a global step counter, which serves as the timeline of the training process. The main training loop is terminated once the step counter has been reached. The outer loop runs multiple training sessions with

the same configurations for reproducibility. Thus, it should be noted that all of the runs use the same hyperparameter selection.

3.2. Experiment Structure and Design

As discussed in the implementation section, multiple hyperparameters must be tuned for each algorithm to perform well. The "REINFORCE" algorithm requires an appropriate network structure, learning rate, and γ . In addition, an appropriate depth needs to be selected for "AC" and "A2C". All algorithms were run on 10^6 environment steps. Due to the nature of "cartpole," γ selection is limited to large values as we are equally interested in distant rewards. Thus, a γ of 0.99 was chosen to be used for all experiments. The learning rate α was set to 0.001 based on prior tuning experiments. This value seemed to consistently produce well-performing models on DQN, thus, it was chosen as the baseline learning rate for the rest of the experiments. This choice implicitly assumes that hyperparameters tuned for value-based methods, like DQN, can reasonably transfer to policy-based methods when in the same environment.

Structure: Due to time and computational constraints, the experimentation follows a progressive structure, where hyperparameters were adjusted incrementally from REINFORCE to AC to A2C. To preserve reproducibility, all experiments were run 5 times with the same hyperparameter configuration. Each experiment was run for exactly 5×10^6 environment steps.

First, the network size and structure were tested on the REINFORCE algorithm. 4 main network architectures were considered. Two with only one hidden layer, each with [64] and [128] neurons respectively, and two with multiple hidden layers [128,64] and [128,64,32] respectively. The performances were then compared both in progression during training and in average rewards throughout training. The best-performing network was then used in the following experiments.

For AC, with the introduction of a critic network, we also have a new hyperparameter in the form of depth referred to in the theory section 2.3 as the n -step return. The network for both actor and critic was set to the best-performing network for the REINFORCE algorithm, and the experimentation here focuses more on the best choice of depth. The three main evaluated values are: 5,15,20. Again, as with reinforcement, the depth value that yielded the highest average reward was selected as the baseline for subsequent A2C experiments.

A2C does not introduce any new hyperparameter; that being said, some adjustments needed to be made for it to

converge properly. Firstly, using the best depth and network obtained from previous testing, two main implementations were evaluated, one with direct advantage update and one with normalized advantage update. Also, it seemed that more performance could be gained for A2C by increasing search depth. Thus, using the best-performing implementation (normalized), two more training depths were tested: $n = 30$ and $n = 50$.

3.3. Experimental results

To measure the performance of evaluated hyperparameter configurations, two main methods were used. The progression over time of each configuration was plotted as the average reward over all 5 runs per episode. This gives us an average trajectory over training steps, describing each configuration's performance progression. The mean of the average trajectory was then estimated for each configuration as a more reliable performance metric. Using the mean to determine progression has the advantage of summarising performance stability, favoring more consistent performers. However, its interpretation is limited to direct comparison rather than a general performance metric, as it reduces the training process to a singular value. In Table 1, the results of the experiments are displayed.

Algorithm	Configuration	Mean Reward
REINFORCE	[64]	386.3
	[128]	399.8
	[128,64]	329.8
	[128,64,32]	301.8
AC [128]	$n=5$	222.0
	$n=15$	243.3
	$n=20$	229.0
A2C [128]	raw adv, $n=20$	174.8
	norm adv, $n=20$	307.7
	norm adv, $n=30$	345.0
	norm adv, $n=50$	390.6

Table 1. Summary of algorithm configurations and their mean rewards. (norm=normalized, adv=advantage, n =depth, []=neuron count)

Looking at the average reward from the above table 1, we can see some configurations perform on average better than the rest. For REINFORCE, the average reward is higher with the shallow [128] network, having the highest reward return at 400. This network was then used for further experimentation. AC seems to perform best on $n=15$ with an average reward of 243. It should be noted that AC seems to have a lower performance than REINFORCE across the board. Comparatively, A2C seems to gain the largest boost to performance when adjusted. With its highest performing configuration taking an average reward value of 390. However, as stated, the average does not tell the whole story. For

a clearer view of progression, we could look at the training performance plotted over environment steps.

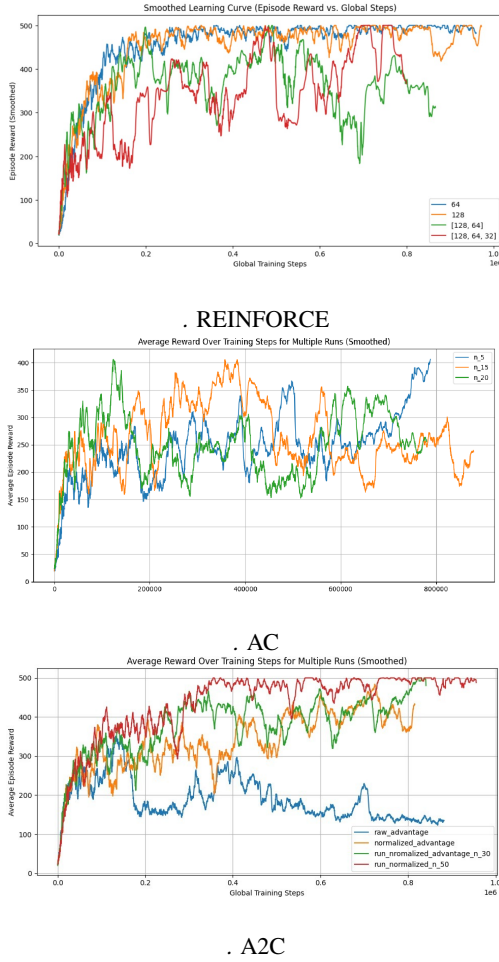


Figure 1. Training progression of configurations for each algorithm.

When evaluating training trajectories as shown in Fig. 1 for REINFORCE, the progression over environment step suggests a stable algorithm when the network size is appropriately chosen. Specifically, the shallower networks outperform the deeper ones both in convergence and stability. We also notice that the larger networks, namely [128,64] and [128,64,32], seem to be unstable and struggle to converge. AC also has this issue. We see that all three configurations vary widely in performance, and even the best-performing one, namely $n=15$, struggles with reaching a stable solution. A2C has the highest varying results when compared to both. Firstly, the configuration without advantage normalization does not perform well. Seemingly, the reward decreases over steps. When looking at the normalized algorithm, they perform increasingly better concerning depth. The configuration with the largest depth $n=50$ seems to be the most stable and yields the highest reward progressively.

4. Discussion

4.1. Discussion of results

From the results obtained after experimentation, REINFORCE outperforms the other implementations. REINFORCE also manages to converge faster than the other implementations. When looking at the plot of the best configurations for each algorithm Fig. 2, as expected, we can see that REINFORCE (blue line) converges and stabilizes much earlier in the training process, stabilizing at around 200,000 environment steps. This implementation shows more stability in training because we are sampling full episodes. In principle, REINFORCE should have the lowest bias and highest variance of the three due to Monte Carlo sampling. However, this variance is not expressed in the results. This could very well be due to the training environment, which is relatively simple and could potentially limit the expression of variance. The same can be said about the appropriate network choice. The shallow networks seem to be outperforming the deep ones. Again, this could very well be task-specific rather than a property of REINFORCE. The shallow networks both have more than enough capacity to approximate the policy. Since shallow networks are easier to train, they are more suitable for this task.

The trajectory of AC suggests that it struggles to stabilize within 10^6 steps. Looking at Fig. 2 we see that AC (orange line) does not manage to converge. AC introduces bias in the form of a critic function. This function attempts to approximate the state value function after a set of n steps. Values beyond this horizon are estimated from the critic network. Using the critic network as estimation allows for multiple updates within one episode; however, as stated, this increases bias as it is a form of bootstrapping. The increase in bias should lead to more stable training compared to REINFORCE; however, again, we are limited by the environment. As stated in REINFORCE, the computational cost of MC sampling is low, and with an implementation such as AC, we introduce quite a large bias in the beginning. The weights of both actor and critic networks are randomized, thus leading to early struggles in training.

A2C at first is the worst-performing one. When raw values of advantage are used the algorithm does not manage to converge meaningfully. This is mainly due to large values of advantage causing exploding gradients, leading to continuous overcorrection and acting similarly to a large learning rate by amplifying the variance. However, after normalization, the algorithm stabilizes and it is almost on par in performance with REINFORCE, even though it does not rely on Monte Carlo sampling. The best performing A2C (green line) configuration seems to be lagging slightly behind REINFORCE as seen in Fig. 2. Again, this early struggle in training seems to be mainly the effect of the early

variance and bias introduced by the critic network. However, A2C manages to strike a balance between the biased nature of AC and the higher variance of REINFORCE. It should be mentioned that A2C is much more sensitive to depth selection, as a smaller n causes the advantage estimation to have more noise, weakening the gradient signal.

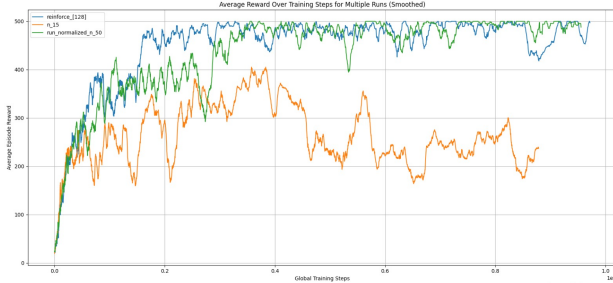


Figure 2. Best configuration of each algorithm

4.2. limitations to our approach

The previous experimentation procedure has a few shortcomings. Firstly, due to the time complexity of each algorithm, a choice had to be made between a maximum number of environment steps and repeated runs on the same configuration. The sample size for each configuration was quite small, leading to inherent uncertainty when estimating the resulting metrics such as the mean reward. This makes it much more difficult to relate experimental results to an exact estimand of algorithmic performance. Another limitation is the environment itself. "Cartpole" is a relatively simple environment; thus, it could lead to underexpression of the properties of an algorithm. For example, with REINFORCE, even though we expect quite a large variance due to the nature of MC sampling, in the cartpole environment, it manages to score consistently high. This leads to a large uncertainty when generalizing conclusions from the cartpole environment. Another more complex environment might be able to give us a clearer view of the limitations of these 3 policy gradient algorithms, allowing us to generalize their performance better.

5. Conclusion

Our results demonstrate that REINFORCE outperforms both AC and A2C on the cartpole environment, given that an appropriate network architecture is chosen. We also notice that the variation of REINFORCE is less than expected; however, as mentioned in the discussion section, this is due to the limited resolution of the cartpole environment. AC struggles significantly to converge to a stable solution. The inherent bias of the actor-critic structure is further emphasized by the added variance in the initialization of the critic network. The bootstrapping procedure then leads to overcorrection

in later training steps. This process is similar to training on a large learning rate. With the addition of Advantage, A2C manages to fix this sharp and early increase in variance, however, this is achieved if the advantage values are adjusted through normalization, otherwise large advantage values have significant leverage over the gradient, leading to large instability and subsequently increasing variance.

The experimentation process has notable limitations. Firstly, the cart pole does not provide us with expensive sampling to warrant bootstrapping methods. Thus unbiased approaches like REINFORCE will yield consistently better solutions. Secondly, the computational time limits the ability for higher resolution tuning, leading us to slightly bias the models that were tested first. For example, AC could benefit from a different learning rate paired up with different network sizes. The same can be said regarding A2C; further adjustments can lead to larger increases in performance. Overall, it seems that when full-episode sampling is computationally feasible, Monte Carlo methods tend to produce more stable and consistent training outcomes. Nonetheless, with appropriate adjustments, A2C achieves competitive performance.